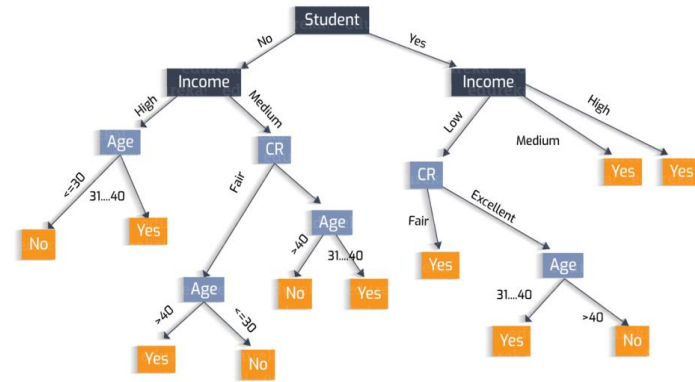


Aprendizaje Supervisado

Arbol de decisión

El aprendizaje basado en **árboles de decisión** está ampliamente extendido en la actualidad, y múltiples modelos hacen diferentes implementaciones de los mismos.

Las primeras versiones de estos modelos fueron implementados por Leo Breiman.
Se utilizan para problemas de Clasificación y Regresión.



Aprenden de los datos generando reglas de tipo **if-else**.

Separan los datos en grupos cada vez más pequeños de subsets de un dataset original.

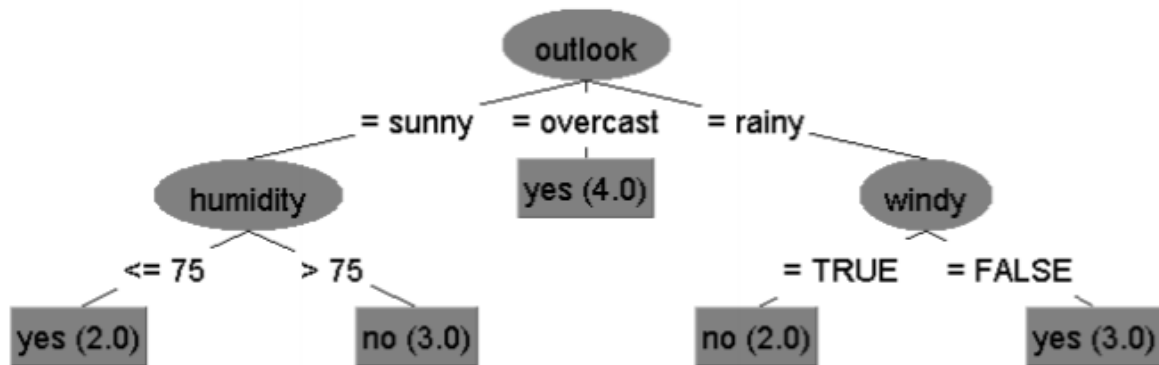
Cada división se la conoce con el nombre de **nodo**. Cuando un nodo no conduce a nuevas divisiones se le denomina **hoja**, para luego ser considerada como ramas del árbol.



Árboles de Decisión:



Gráfico 1. Mejor clima para jugar tenis



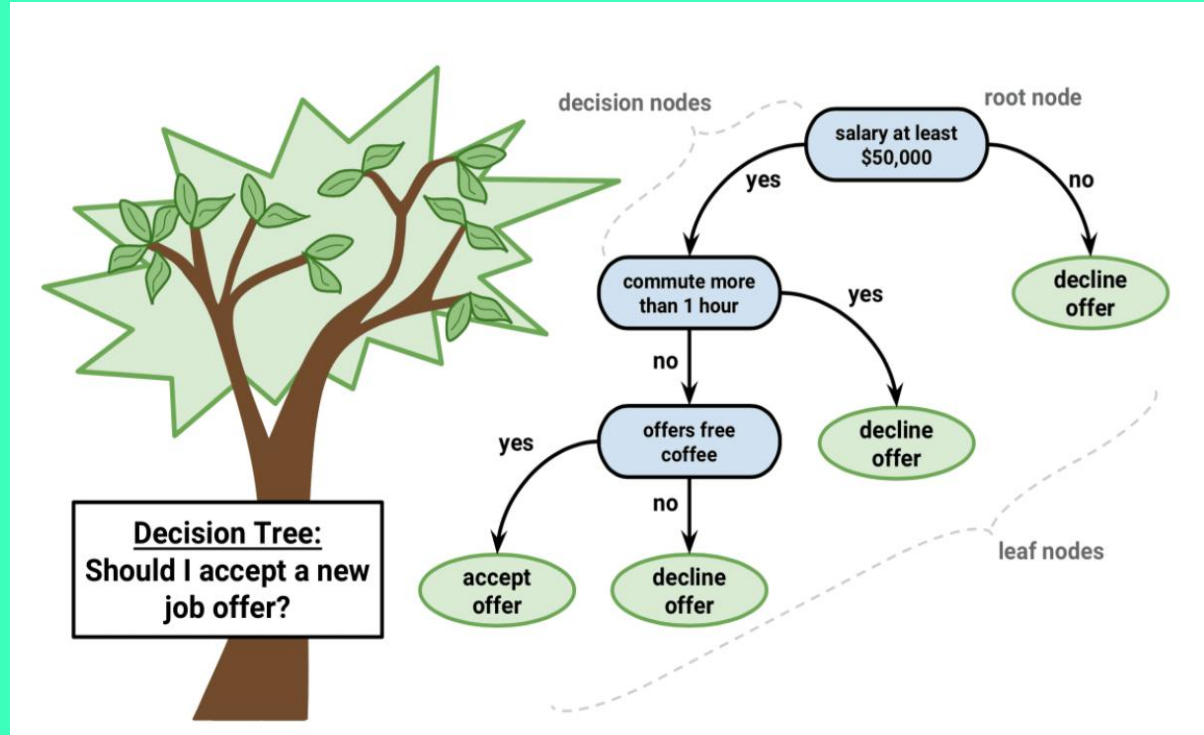
Partes de los Árboles de Decisión:

- **Nodo raíz:** Representa a toda la población o muestra y esto se divide en dos o más conjuntos homogéneos.
- **División:** Es un proceso de división de un nodo en dos o más subnodos.
- **Nodo de decisión:** Cuando un subnodo se divide en subnodos adicionales, se llama nodo de decisión.
- **Nodo de hoja / terminal:** Los nodos sin hijos (sin división adicional) se llaman Hoja o nodo terminal.

Partes de los Árboles de Decisión:

- **Poda (Pruning):** Consiste en la reducción del tamaño de los árboles de decisión eliminando nodos.
- **Rama / Subárbol:** Una subsección del árbol de decisión se denomina rama o subárbol.
- **Nodo padre e hijo:** Un nodo, que se divide en subnodos se denomina nodo principal de subnodos, mientras que los subnodos son hijos de un nodo principal.

Partes de los Árboles de Decisión:



¿Cuáles son las ventajas?

- Caja blanca (conjunto de reglas con booleanos), **sus resultados son fáciles de entender e interpretar.**
- Relativamente robusto.
- Funcionan relativamente bien con grandes conjuntos de datos.
- **Combinaciones de los mismos pueden dar resultados muy certeros sin perder explicabilidad**, por ejemplo, Random Forest.

¿Cuáles son las desventajas?

- **Tienden al sobreajuste u overfitting de los datos**, por lo que el modelo al predecir nuevos casos no estima con el mismo índice de acierto.
- **Se ven influenciadas por los outliers**, creando árboles con ramas muy profundas que no predicen bien para nuevos casos.
- Crear árboles demasiado complejos puede conllevar que no se adapten bien a los nuevos datos.
- Se pueden **crear árboles sesgados** si una de las clases es más numerosa que otra es decir, si hay desbalance de clases.



```
import pandas as pd
import numpy as np

# Generar un dataset ficticio
np.random.seed(42)

# Definir posibles valores para las características
tipos_fruta = ['manzana', 'banana', 'naranja', 'pera', 'uva']
temporadas = ['primavera', 'verano', 'otoño', 'invierno']
regiones = ['norte', 'sur', 'este', 'oeste']

# Generar datos aleatorios
num_datos = 100
tipos_fruta_data = np.random.choice(tipos_fruta, num_datos)
temporadas_data = np.random.choice(temporadas, num_datos)
regiones_data = np.random.choice(regiones, num_datos)
precios_data = np.random.uniform(1, 10, num_datos) # Precios aleatorios entre 1 y 10

# Crear el DataFrame
datos_ficticios = pd.DataFrame({
    'tipo_fruta': tipos_fruta_data,
    'temporada': temporadas_data,
    'region': regiones_data,
    'precio': precios_data
})
```



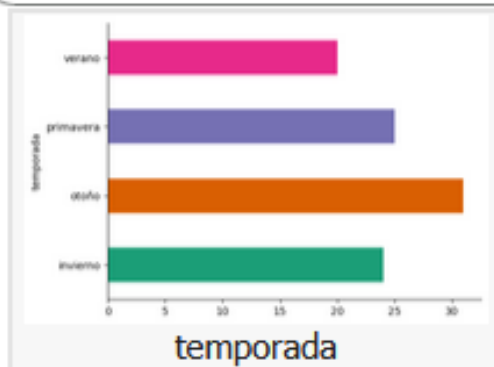
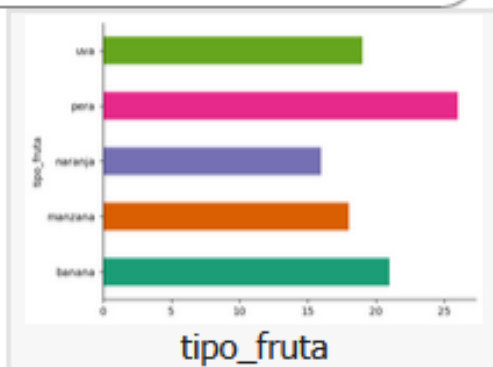
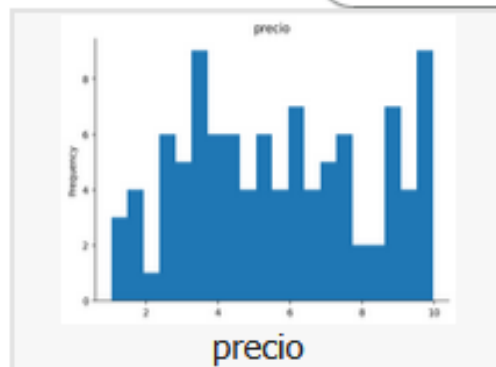
	tipo_fruta	temporada	region	precio
0	pera	invierno	este	7.081211
1	uva	otoño	norte	7.616945
2	naranja	primavera	sur	2.881645
3	uva	invierno	norte	5.873032
4	uva	invierno	norte	7.262060



Pasos siguientes:

[Generar código con datos_ficticios](#)

[Ver gráficos recomendados](#)



```
from sklearn.tree import DecisionTreeRegressor
from sklearn.model_selection import train_test_split
from sklearn import tree
import matplotlib.pyplot as plt

# Cargar datos
datos = pd.read_csv('precios_frutas.csv')

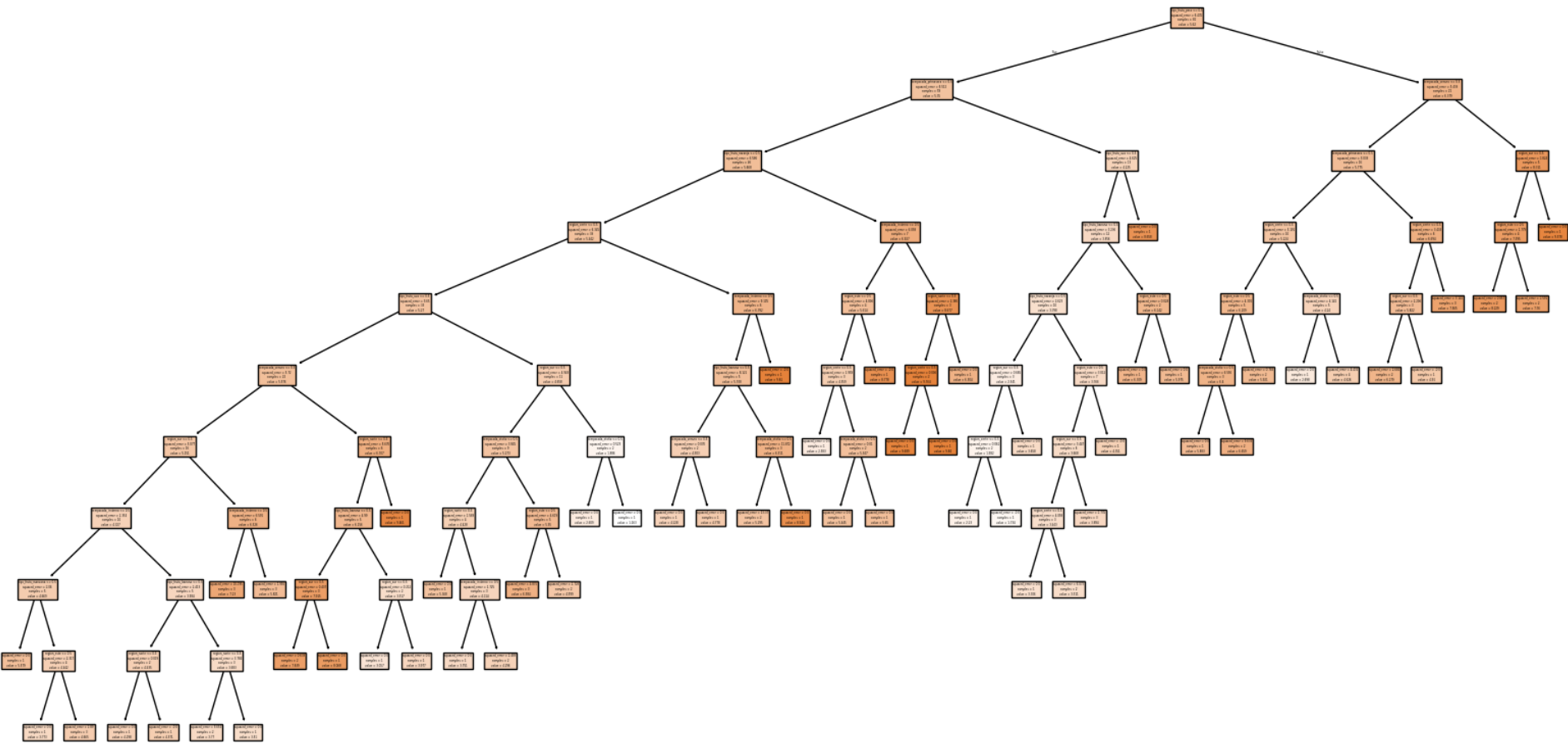
# Selección de características y variable objetivo
X = datos[['tipo_fruta', 'temporada', 'region']]
y = datos['precio']

# Codificación de variables categóricas
X = pd.get_dummies(X)

# División de datos en entrenamiento y prueba
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Entrenamiento del modelo
modelo = DecisionTreeRegressor()
modelo.fit(X_train, y_train)

# Visualización del árbol
plt.figure(figsize=(20,10))
tree.plot_tree(modelo, feature_names=X.columns, filled=True)
plt.show()
```



```
# Selección de características y variable objetivo
X = datos[['tipo_fruta', 'temporada', 'region']]
y = datos['precio']
```

1.X (features / variables independientes)

- datos[['tipo_fruta', 'temporada', 'region']]
- Aquí se seleccionan tres columnas del DataFrame:
 - tipo_fruta: por ejemplo, "manzana", "pera", "mango".
 - temporada: estación del año o temporada de cosecha.
 - region: zona o lugar de origen de la fruta.
- Estas son las **variables explicativas** que se usarán como entrada al modelo.

2.y (target / variable dependiente)

- datos['precio']
- Aquí se selecciona la columna que contiene el **valor que queremos predecir**: el **precio** de la fruta.
- Es la **variable objetivo** del modelo.

En otras palabras:

- X = la información con la que el modelo intentará aprender patrones.
- y = el resultado que queremos que el modelo prediga.

```
# Codificación de variables categóricas
```

```
X = pd.get_dummies(X)
```

pd.get_dummies()

- Función de Pandas que toma columnas con texto (categóricas) y crea nuevas columnas binarias (0/1) para cada categoría.
- Esto se llama **One-Hot Encoding**.

1. Entrada (X antes de get_dummies):

tipo_fruta	temporada	region
Manzana	Verano	Norte
Pera	Invierno	Sur
Mango	Verano	Centro
Manzana	Otoño	Norte
Pera	Primavera	Sur

Salida (X después de get_dummies):

tipo_frut a_Manzana	tipo_frut a_Pera	tipo_frut a_Mango	tempora da_Verano	tempora da_Invierno	tempora da_Otoño	tempora da_Primavera	region_ Norte	region_ Sur	region_ Centro
1	0	0	1	0	0	0	1	0	0
0	1	0	0	1	0	0	0	1	0
0	0	1	1	0	0	0	0	0	1
1	0	0	0	0	1	0	1	0	0
0	1	0	0	0	0	1			

Lo que logra:

- Convierte texto en números sin asignar valores arbitrarios (como 1, 2, 3) que podrían confundir al modelo haciéndole pensar que hay un orden o distancia entre categorías.
- Permite que cada categoría sea representada como una característica independiente.

```
# División de datos en entrenamiento y prueba
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

train_test_Split

- Función de scikit-learn que mezcla y divide datos en dos o más partes.
- Recibe como entrada:
 - X: características (variables independientes).
 - y: variable objetivo (lo que quieres predecir).

Parámetros importantes:

•test_size=0.2

- Significa que el **20%** de los datos será para **prueba** y el **80%** para **entrenamiento**.

•random_state=42

- Fija una "semilla" para que la división sea siempre igual cuando vuelvas a ejecutar el código (reproducibilidad).
- Si no lo pones, cada ejecución te dará una división distinta.

```
# División de datos en entrenamiento y prueba
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

Qué produce:

- X_train: características del conjunto de entrenamiento (80%).
- X_test: características del conjunto de prueba (20%).
- y_train: variable objetivo correspondiente a X_train.
- y_test: variable objetivo correspondiente a X_test.

random_state=42 asegura que la división de datos sea **siempre igual**, útil para comparar resultados entre ejecuciones o cuando otros ejecutan tu mismo código.

```
# División de datos en entrenamiento y prueba
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

Cuando `train_test_split` divide los datos, **los mezcla de forma aleatoria** antes de separar en entrenamiento y prueba.

Si no fijas `random_state`, cada vez que ejecutes el código la división será distinta.

Ejemplo sin `random_state`:

- Primera ejecución → registros [1, 5, 8, 2, ...] en entrenamiento.
- Segunda ejecución → registros [4, 1, 7, 9, ...] en entrenamiento.

¿Qué hace `random_state`?

- Es un número entero (puede ser cualquiera: 0, 1, 42, 1234...).
- Fijarlo significa que **la aleatoriedad será siempre la misma**, por lo que la división de datos será idéntica en todas las ejecuciones.
- Esto se llama **reproducibilidad**.

Si el dataset tiene:

fruta	temporada	región	precio
1	A	X	3.5
2	B	Y	4.0
3	C	X	5.2
4	A	Z	3.8
5	B	Y	4.1
6	C	Z	3.7
7	A	X	4.5
8	B	Y	3.9
9	C	X	4.6
10	A	Z	3.4

Con test_size=0.2 →

- **Entrenamiento:** 8 registros (X_train, y_train).
- **Prueba:** 2 registros (X_test, y_test).

```
# Entrenamiento del modelo
modelo = DecisionTreeRegressor()
modelo.fit(X_train, y_train)
```

modelo = DecisionTreeRegressor()

- Esto crea una **instancia** de un árbol de decisión para **regresión** (no clasificación).
- En esta línea no se entrena nada todavía, solo se configura el modelo.
- Puedes pasarle parámetros opcionales, por ejemplo:
 - **max_depth**: para limitar la profundidad del árbol.
 - **min_samples_split**: para controlar cuántas muestras se necesitan para dividir un nodo.
 - **random_state**: para reproducir siempre el mismo árbol.

```
# Entrenamiento del modelo
modelo = DecisionTreeRegressor()
modelo.fit(X_train, y_train)
```

modelo.fit(X_train, y_train)

- Aquí sí ocurre el **entrenamiento**:

- El modelo analiza los datos de entrada (X_train) y los valores reales (y_train).
- Va buscando divisiones (splits) en las variables que reduzcan el error de predicción.
- Construye un árbol de nodos y hojas donde:
- El resultado final es un árbol entrenado que puede **predecir el precio** de una fruta nueva dadas sus características.

```
# Visualización del árbol
plt.figure(figsize=(20,10))
tree.plot_tree(modelo, feature_names=X.columns, filled=True)
plt.show()
```

plt.figure(figsize=(20, 10))

- Crea una nueva figura para el gráfico.
- **figsize=(20, 10)** define el tamaño del gráfico en pulgadas (ancho=20, alto=10).
- Esto es importante porque los árboles pueden ser grandes, y sin ajustar el tamaño podrían verse amontonados.

tree.plot_tree(modelo, feature_names=X.columns, filled=True)

- Dibuja el árbol usando el modelo ya entrenado (modelo).
- **feature_names=X.columns**: usa los nombres originales de las variables (tipo_fruta_Manzana, region_Sur, etc.) en lugar de índices como 0, 1, 2.
- **filled=True**: rellena los nodos con colores que representan los valores (más oscuro o más claro según el valor de la predicción en ese nodo).
- El gráfico muestra:
 - **Condiciones de división** en cada nodo (ej. $\text{region_Sur} \leq 0.5$).
 - **Número de muestras** que llegan a ese nodo.
 - **Valor promedio** de la variable objetivo para ese nodo.

plt.show()

- Muestra el gráfico en pantalla.

```
from sklearn.tree import DecisionTreeRegressor
from sklearn.model_selection import train_test_split
from sklearn import tree
import matplotlib.pyplot as plt

# Cargar datos
datos = pd.read_csv('precios_frutas.csv')

# Selección de características y variable objetivo
X = datos[['tipo_fruta', 'temporada', 'region']]
y = datos['precio']

# Codificación de variables categóricas
X = pd.get_dummies(X)

# División de datos en entrenamiento y prueba
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Entrenamiento del modelo
modelo = DecisionTreeRegressor()
modelo.fit(X_train, y_train)

# Visualización del árbol
plt.figure(figsize=(20,10))
tree.plot_tree(modelo, feature_names=X.columns, filled=True)
plt.show()
```

```
from sklearn.tree import export_text

tree_rules = export_text(modelo, feature_names=list(X.columns))
print(tree_rules)
```

```
tree_rules = export_text(modelo, feature_names=list(X.columns))
print(tree_rules)
```

[illegible]

Uso e Importancia de los Árboles de Decisión



Los árboles de decisión **simulan un proceso de toma de decisiones** basado en características clave de los datos. Para predecir el precio de una fruta, el árbol hace preguntas como:



- ✓ ¿Es verano? → Si **sí**, sigue un camino distinto al de **no**.
- ✓ ¿La fruta es importada o local? → Toma otro camino.
- ✓ ¿Se produce en la región norte o sur? → Otro factor que afecta el precio.



Esto permite al modelo capturar patrones en los datos sin requerir mucha ingeniería manual.

Uso e Importancia de los Árboles de Decisión

♦ Beneficios en este caso

1. **Fácil de interpretar** → Podemos ver qué factores afectan más al precio.
2. **Maneja datos categóricos y numéricos** → Puede trabajar con variables como "temporada" y "región" sin problemas.
3. **Detecta interacciones entre variables** → Por ejemplo, si el precio cambia en función de la combinación de "temporada" y "tipo de fruta".
4. **No necesita mucha normalización de datos** → A diferencia de modelos como regresión lineal o redes neuronales.

Malos Usos y Limitaciones



Aunque los árboles de decisión son útiles, tienen debilidades que pueden llevar a malos resultados.



1. Sobreajuste (Overfitting) Si no se limita la profundidad del árbol, este puede volverse demasiado específico con los datos de entrenamiento y no generalizar bien a nuevos datos.



Ejemplo: Si el árbol tiene 50 niveles de profundidad, memorizará datos individuales en lugar de encontrar patrones generales. Solución → Limitar la profundidad ($\text{max_depth}=5$ o $\text{max_depth}=10$) o usar técnicas como poda (pruning).

2. Sensibilidad a Datos de Entrenamiento



Los árboles de decisión son muy sensibles a los datos con los que se entrenan. Si los datos están desbalanceados o tienen ruido, el modelo puede generar reglas incorrectas.



Ejemplo: Si hay pocas observaciones de naranjas en invierno, el modelo puede predecir mal su precio porque no tiene suficiente información.



Solución → Asegurar que el dataset es representativo y balanceado.

3. No Captura Relaciones Lineales Tan Bien

Si el precio de las frutas sigue una **relación lineal clara** con factores como la temporada y la oferta/demanda, un **modelo de regresión lineal** podría ser más adecuado.

Ejemplo:

Si el precio de una fruta aumenta en **\$0.50 por cada 10% de reducción en la oferta**, un árbol podría no capturar esto de manera eficiente.

Solución → Comparar con otros modelos como regresión lineal o Random Forest.

4. No Maneja Bien Variables Muy Dependientes

Si dos variables están altamente correlacionadas, un árbol puede enfocarse demasiado en una de ellas e ignorar la otra.

Ejemplo: Si `region` y `tipo_fruta` están muy relacionados (porque ciertas frutas solo crecen en ciertas regiones), el árbol podría usar una sola de ellas y perder información valiosa.

Solución → Revisar correlaciones antes de entrenar el modelo y hacer pruebas con diferentes conjuntos de variables.

CLASIFICACION

```
import pandas as pd
import numpy as np

# Generar un dataset ficticio
np.random.seed(42)

# Definir posibles valores para las características
tipos_fruta = ['manzana', 'banana', 'naranja', 'pera', 'uva']
temporadas = ['primavera', 'verano', 'otoño', 'invierno']
regiones = ['norte', 'sur', 'este', 'oeste']

# Generar datos aleatorios
num_datos = 100
tipos_fruta_data = np.random.choice(tipos_fruta, num_datos)
temporadas_data = np.random.choice(temporadas, num_datos)
regiones_data = np.random.choice(regiones, num_datos)
precios_data = np.random.uniform(1, 10, num_datos) # Precios aleatorios entre 1 y 10

# Crear el DataFrame
datos_ficticios = pd.DataFrame({
    'tipo_fruta': tipos_fruta_data,
    'temporada': temporadas_data,
    'region': regiones_data,
    'precio': precios_data
})

# Guardar el dataset ficticio en un archivo CSV para usarlo en el código original
datos_ficticios.to_csv('precios_frutas.csv', index=False)
```

```
# Crear etiqueta de clasificación: precio mayor a 4.0 es "Cara", si no es "Barata"
datos['categoria_precio'] = ['Cara' if p > 4.0 else 'Barata' for p in datos['precio']]

# Variables de entrada y salida
columnas_categoricas = ['tipo_fruta', 'temporada', 'region']
X = datos[columnas_categoricas]
y = datos['categoria_precio']

# Transformador para variables categóricas
transformador = ColumnTransformer(
    transformers=[
        ('cat', OneHotEncoder(handle_unknown='ignore'), columnas_categoricas)
    ],
    remainder='drop'
)

# Codificación
X_codificado = transformador.fit_transform(X)

# División de datos
X_train, X_test, y_train, y_test = train_test_split(
    X_codificado, y, test_size=0.2, random_state=42
)

# Entrenamiento
modelo = DecisionTreeClassifier(random_state=42)
modelo.fit(X_train, y_train)
```

```
# Variables de entrada y salida
columnas_categoricas = ['tipo_fruta', 'temporada', 'region']
X = datos[columnas_categoricas]
y = datos['categoria_precio']
|
```

.Variables de entrada (X) y salida (y)

- **columnas_categoricas**: lista con los nombres de las columnas que se usarán como **features** (características predictoras).
- **X**: subconjunto del DataFrame datos que contiene **solo** esas columnas.
- **y**: la columna objetivo (categoria_precio) que queremos predecir.

```
# Crear etiqueta de clasificación: precio mayor a 4.0 es "Cara", si no es "Barata"
datos['categoria_precio'] = ['Cara' if p > 4.0 else 'Barata' for p in datos['precio']]
```

Qué hace:

Crea una nueva columna en el DataFrame datos llamada categoria_precio.

Si el precio (p) es mayor a 4.0, se asigna la etiqueta "Cara".

Si no, se asigna "Barata".

Ejemplo:

precio	categoria_precio
5.3	Cara
3.7	Barata
4.1	Cara

```

1
# Transformador para variables categóricas
transformador = ColumnTransformer(
    transformers=[
        ('cat', OneHotEncoder(handle_unknown='ignore'), columnas_categoricas)
    ],
    remainder='drop'
)

# Codificación
X_codificado = transformador.fit_transform(X)

```

•Qué es:

Un **ColumnTransformer** es una herramienta de scikit-learn para aplicar transformaciones distintas a columnas específicas.

•Partes:

•('cat', OneHotEncoder(handle_unknown='ignore'), columnas_categoricas):

•**cat**: nombre que le damos a este transformador (interno).

•**OneHotEncoder**: convierte variables categóricas en variables binarias (one-hot encoding).

•Ejemplo:

region = "Norte" → region_Norte=1, region_Sur=0, region_Este=0

•**handle_unknown='ignore'**: evita error si aparece una categoría nueva que no estaba en entrenamiento.

•**columnas_categoricas**: columnas a las que se aplicará el OneHotEncoder.

•**remainder='drop'**: indica que todas las demás columnas (no listadas en columnas_categoricas) se eliminan.

```
# Entrenamiento
```

```
modelo = DecisionTreeClassifier(random_state=42)
```

```
modelo.fit(X_train, y_train)
```

```
# Evaluación
```

```
print(f"Precisión en entrenamiento: {modelo.score(X_train, y_train):.2f}")
```

```
print(f"Precisión en prueba: {modelo.score(X_test, y_test):.2f}")
```

```
# Nombres de columnas codificadas
```

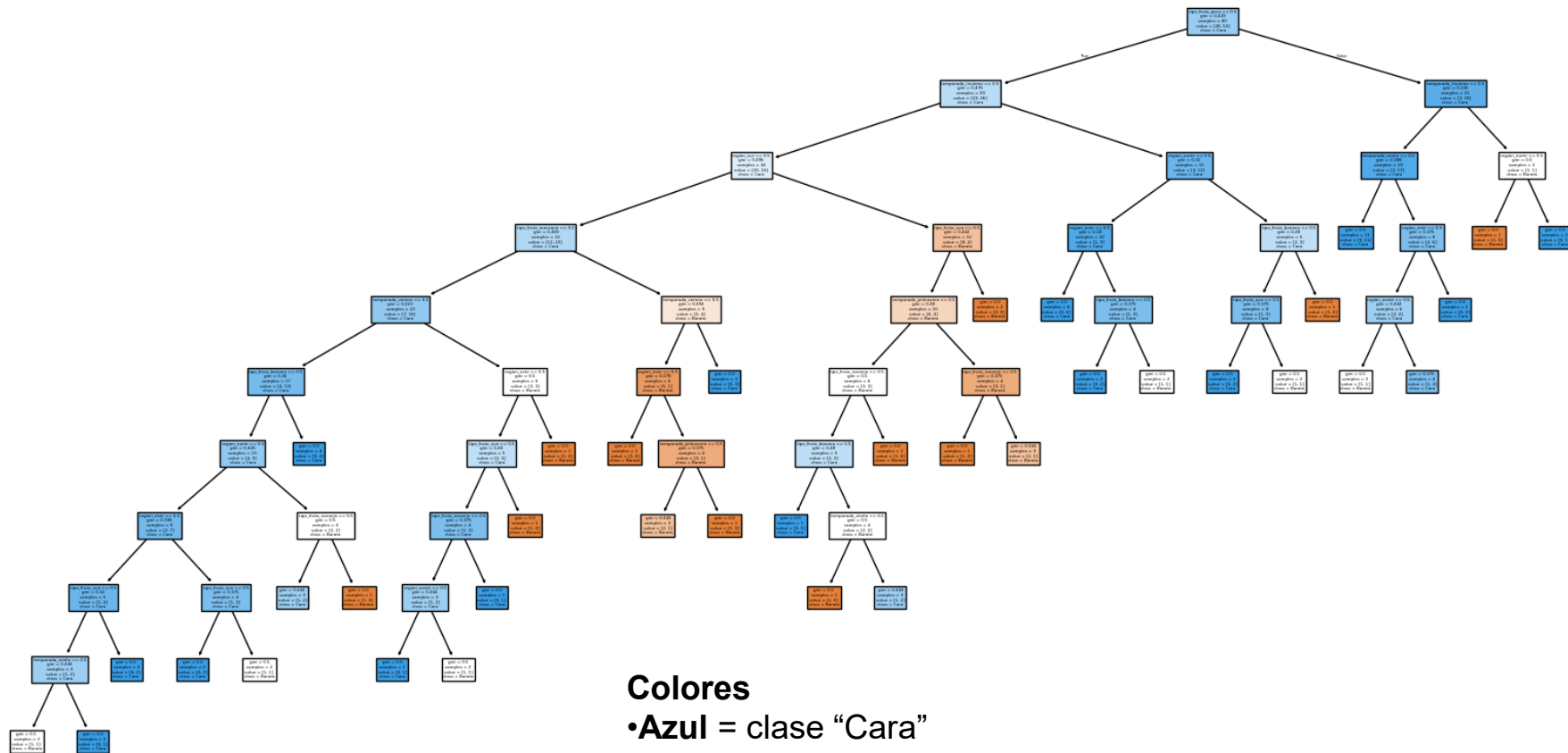
```
nombres_columnas = transformador.named_transformers_['cat'].get_feature_names_out(columnas_categoricas)
```

```
# Visualización del árbol
```

```
plt.figure(figsize=(20, 10))
```

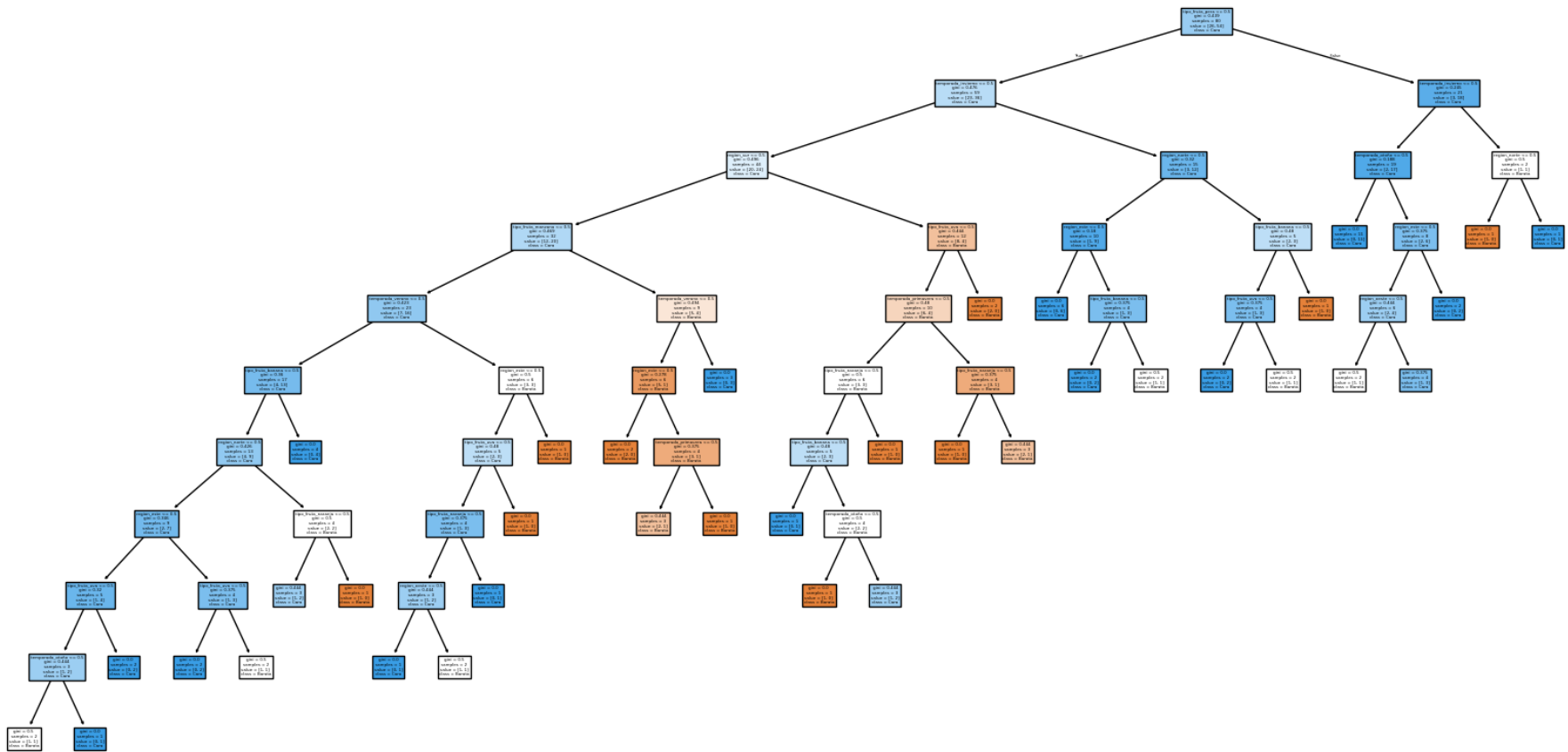
```
tree.plot_tree(modelo, feature_names=nombres_columnas, class_names=modelo.classes_, filled=True)
```

```
plt.show()
```



Colores

- **Azul** = clase “Cara”
- **Naranja** = clase “Barata”
- **Más intenso** = mayor pureza (más seguro el modelo de que ahí domina una clase).
- **Más claro** = más mezcla de clases (menos certeza).



Ejemplo de lectura de una ruta:

1. $\text{tipo_fruta_Manzana} \leq 0.5 \rightarrow$ si es Manzana (1) vas por la derecha, si no por la izquierda.
2. $\text{region_Norte} \leq 0.5 \rightarrow$ si está en Norte (1) vas por la derecha, si no por la izquierda.

Los árboles de decisión **son una herramienta poderosa** para predecir el precio de frutas, ya que pueden capturar relaciones complejas y reglas de decisión. Sin embargo, **hay que usarlos correctamente** para evitar problemas como sobreajuste o dependencia excesiva de los datos de entrenamiento.

- Para aplicar bien los árboles en este contexto:
 - ✓ Ajustar la **profundidad** para evitar sobreajuste.
 - ✓ Comparar con **otros modelos** como Random Forest o regresión lineal.
 - ✓ Revisar la **importancia de las variables** para optimizar el modelo.

Comparativa Árbol de Regresión vs. Árbol de Clasificación

Aspecto	Árbol de Regresión	Árbol de Clasificación
Objetivo	Predecir un valor numérico continuo (ej. precio, temperatura, peso).	Predecir una etiqueta o clase (ej. fruta “Cara” o “Barata”, tipo de animal).
Algoritmo en scikit-learn	DecisionTreeRegressor	DecisionTreeClassifier
Variable objetivo (y)	Valores numéricos (float, int).	Categorías (str, int codificado).
Salida en las hojas	Promedio de los valores de y en ese nodo.	Clase más frecuente en ese nodo.
Métrica por defecto en .score()	R^2 (coeficiente de determinación).	Accuracy (proporción de aciertos).
Visualización (plot_tree)	Valores promedio numéricos.	Nombre de la clase y proporción de muestras.
Ejemplo de uso	Predecir el precio de una fruta.	Predecir si una fruta es “Cara” o “Barata”.

TALLER

¡MUCHAS GRACIAS!

